Crashlytics Operations Runbook

This document describes how the operator (or any authorized human) inspects, triages, and closes Crashlytics issues for Lava. The Firebase CLI has no `crashlytics:list` command, and the Crashlytics REST API is not publicly available — Console + BigQuery export are the two supported paths.

Quick: viewing crashes

1. Open the Crashlytics Console: https://console.firebase.google.com/project/lava-vasic-digital/crashlytics
2. The "Issues" tab lists every distinct crash signature with:
   - **Title** — the top frame of the stack trace
   - **Affected users** + **Affected sessions**
   - **First seen** + **Last seen** + **Latest version** affected
3. Filter by version: top-of-page filter dropdown — pick `1.2.5 (1025)` (or whatever current build) to scope the issue list to the live version.
4. Click an issue → "Stack trace" tab → expand any session → see the deobfuscated stack trace, custom keys (build_type, version_name, version_code, application_id), breadcrumbs, and device info.

Closing a Crashlytics issue (per §6.O clause 5)

§6.O Crashlytics-Resolved Issue Coverage Mandate requires the closure to happen AFTER the test coverage lands, never before:

```
1. Land the fix commit + the validation test + the Challenge Test
+
   the closure log at .lava-ci-evidence/crashlytics-resolved/
<date>-<slug>.md
2. Distribute the fixed build via ./scripts/distribute.sh (auto-
bumps versionCode)
3. Wait at least 24h for tester usage to confirm the fix
4. In the Crashlytics Console, click the issue → "Close issue"
button
5. In the close-comment, paste the closure log path so future
reviewers
   can find the post-mortem
```

BigQuery export (recommended for advanced queries)

Crashlytics writes every event to BigQuery if you enable the export. One-time setup:

1. Console → Crashlytics → "Settings" (gear icon) → "BigQuery export" → toggle ON.
2. Choose dataset: `firebase_crashlytics`.
3. After ~24h, query via `gcloud` or the BigQuery web console:

```
-- Top 10 crash signatures in the last 7 days
SELECT
  issue_title,
  COUNT(*) AS occurrences,
  COUNT(DISTINCT installation_uuid) AS affected_users
FROM `lava-vasic-digital.firebase_crashlytics.android_*`
WHERE event_timestamp > TIMESTAMP_SUB(CURRENT_TIMESTAMP(),
        INTERVAL 7 DAY)
  AND is_fatal = TRUE
GROUP BY issue_title
ORDER BY occurrences DESC
LIMIT 10;

-- All crashes for a specific version
SELECT issue_title, exception_type, blame_frame_file,
        blame_frame_line
FROM `lava-vasic-digital.firebase_crashlytics.android_*`
WHERE app_display_version = '1.2.5'
  AND event_timestamp > TIMESTAMP_SUB(CURRENT_TIMESTAMP(),
        INTERVAL 24 HOUR);
```

Live status check via gcloud (lightweight)

If BigQuery is exporting, a quick "any new crashes since last build?" check:

```
gcloud auth login
PROJECT=lava-vasic-digital
gcloud config set project "$PROJECT"

# Are any fatal crashes recorded in the last hour?
bq query --use_legacy_sql=false --format=prettyjson \
  "SELECT COUNT(*) AS cnt FROM \`$
        {PROJECT}.firebase_crashlytics.android_*\`
   WHERE event_timestamp > TIMESTAMP_SUB(CURRENT_TIMESTAMP(),
        INTERVAL 1 HOUR)
     AND is_fatal = TRUE"
```

Related project files

- `.lava-ci-evidence/crashlytics-resolved/` — closure logs (one per resolved issue)
- `app/src/main/kotlin/digital/vasic/lava/client/firebase/FirebaseInitializer.kt` — defensive init, custom keys
- `app/src/test/kotlin/digital/vasic/lava/client/firebase/FirebaseInitializerTest.kt` — validation
- `app/src/androidTest/kotlin/lava/app/challenges/Challenge13FirebaseColdStartResilienceTest.kt` — Challenge Test
- `scripts/firebase-stats.sh` — prints all dashboard URLs
- `docs/FIREBASE.md` — full Firebase architecture

Constitutional bindings

- **§6.O Crashlytics-Resolved Issue Coverage Mandate** — every closed issue requires (a) validation test, (b) Challenge Test, (c) closure log
- **§6.P Distribution Versioning + Changelog Mandate** — every distribute action MUST bump versionCode + add a CHANGELOG entry
- **§6.J / §6.L Anti-Bluff** — fixes target root causes, not symptoms; tests verify user-visible state